

Q3 (a) Explain the following with respect to State Chart Diagram. [7 Marks]

(SPPU BE OOMD – Simple Answer)

Introduction

A **State Chart Diagram** (State Machine Diagram) is a UML behavioral diagram that represents the **different states of an object and transitions between those states** in response to events.

The following are important concepts of State Chart Diagram:

i) Composite State

Definition

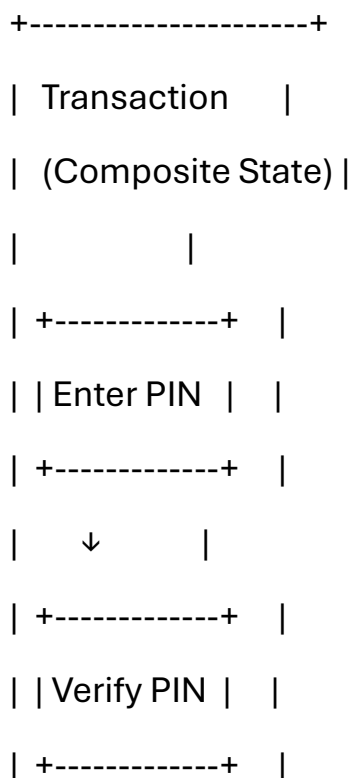
A **Composite State** is a state that contains **one or more substates inside it**. It is used to represent complex behavior by dividing a large state into smaller states.

A composite state helps to simplify complex state diagrams and improves readability.

Types of Composite State:

1. **Sequential Composite State**
2. **Concurrent Composite State**

Example: ATM System



+-----+

Here, **Transaction** is a composite state containing smaller states:

- Enter PIN
- Verify PIN

Advantages:

- Reduces complexity.
 - Organizes states hierarchically.
 - Makes state diagrams easier to understand.
-

ii) Self Transition

Definition

A **Self Transition** is a transition in which an object moves from a state back to the **same state** after an event occurs.

The state remains the same, but an action is performed.

Notation:

+-----+

| State A |

+-----+

↻

Event

Example:

ATM System:

State: Waiting for PIN

Event: Invalid PIN entered

Waiting for PIN —————→ Waiting for PIN

The ATM remains in the **Waiting for PIN** state but increases the attempt count.

Advantages:

- Represents repeated actions.
 - Useful for handling errors or repeated events.
-

iii) Orthogonal States

Definition

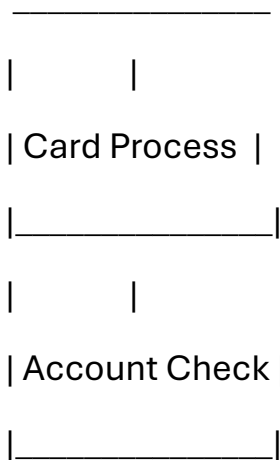
Orthogonal States are states that allow an object to be in **multiple states simultaneously**. They represent parallel or concurrent activities.

An orthogonal state is divided into multiple regions, and each region operates independently.

Example: ATM System

During a transaction:

Transaction

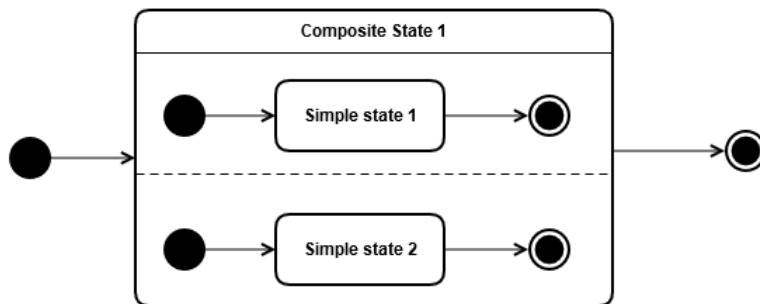
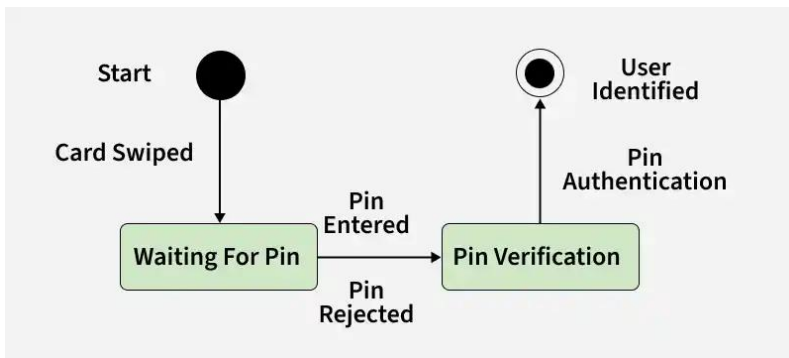


Both activities can occur at the same time:

- Card verification
- Account verification

Advantages:

- Represents parallel behavior.
- Models concurrent activities.
- Reduces complexity in large systems.



Q3 (b) Draw State Diagram with Composite States for ATM Card Reading & Authentication System. [6 Marks]

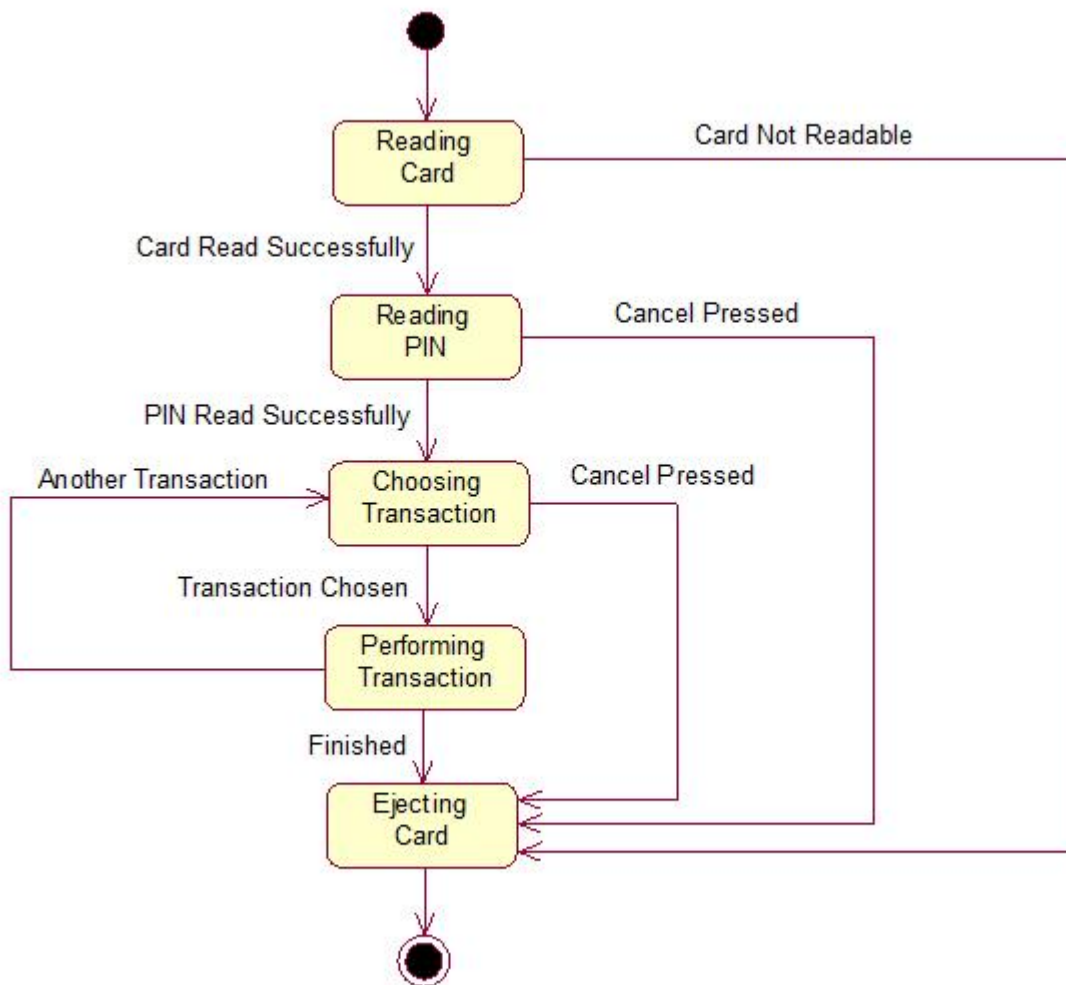
(SPPU BE OOMD – Simple Answer)

Definition

A **State Diagram** (State Chart Diagram) represents the **different states of an object and the transitions between those states** based on events. It is used to model the dynamic behavior of a system.

In an **ATM Card Reading & Authentication System**, the **Authentication** process is represented as a **Composite State** because it consists of multiple substates.

State Diagram



Explanation

1. Initial State

- The ATM is waiting for the customer to insert the card.

2. Read Card

- The ATM reads the information stored on the ATM card.

3. Composite State – Authentication

The **Authentication** state is a **Composite State** because it contains two substates:

- **Enter PIN**
- **Verify PIN**

4. PIN Verification

- If the PIN is **correct**, the customer is taken to the **Transaction Menu**.

- If the PIN is **incorrect**, the system returns to **Enter PIN** and asks the user to try again.

5. Transaction Menu

- The customer performs operations such as **Withdraw**, **Deposit**, or **Balance Inquiry**.

6. Eject Card

- After completing the transaction, the ATM ejects the card.

7. Final State

- The ATM session ends.

Advantages of State Diagram

- Shows the dynamic behavior of the ATM system.
- Represents different states and transitions clearly.
- Makes system behavior easy to understand.
- Helps in software design and testing.

Conclusion

The ATM state diagram models the complete flow from **card insertion** to **authentication**, **transaction processing**, and **card ejection**. The **Authentication** state is modeled as a **Composite State** because it contains the substates **Enter PIN** and **Verify PIN**.

Q3 (c) Explain the following terms:

i) Aggregation

ii) Composition [5 Marks]

(SPPU BE OOMD – Simple Answer)

i) Aggregation

Definition

Aggregation is a **special type of Association** that represents a **weak "Has-A" relationship** between two classes. In aggregation, the child object **can exist independently** of the parent object.

It represents a **whole-part relationship**, where removing the whole does not destroy the parts.

UML Notation

Department ◇----- Professor

(◇ = *Hollow Diamond*)

Example

- A **Department** has many **Professors**.
- If the **Department** is closed, the **Professors** still exist and can work in another department.

Characteristics

- Weak "Has-A" relationship.
- Child object exists independently.
- Represented by a **hollow diamond (◇)**.

Advantages

- Promotes object reusability.
- Reduces dependency between objects.
- Makes the system more flexible.

ii) Composition

Definition

Composition is a **strong form of Aggregation** that represents a **strong "Has-A" relationship**. In composition, the child object **cannot exist without the parent object**. If the parent object is destroyed, the child object is also destroyed.

UML Notation

House ◆----- Room

(◆ = *Filled Diamond*)

Example

- A **House** contains **Rooms**.
- If the **House** is demolished, the **Rooms** also cease to exist.

Characteristics

- Strong "Has-A" relationship.
- Child object depends on the parent.
- Represented by a **filled diamond (◆)**.

Advantages

- Ensures strong ownership.

- Improves data integrity.
- Models real-life ownership relationships effectively.

Q4 (a) Explain different components of a State Diagram. [5 Marks]

(SPPU BE OOMD – Simple Answer)

Definition

A **State Diagram (State Chart Diagram)** is a UML behavioral diagram that represents the **different states of an object and the transitions between those states** in response to events during its lifetime.

The main components of a State Diagram are:

1. Initial State
2. State
3. Transition
4. Event
5. Final State

1. Initial State

Definition

The **Initial State** is the **starting point** of the state diagram. It indicates where the object begins its execution.

UML Notation



Example

In an ATM system, the initial state is "**Waiting for Card**".

2. State

Definition

A **State** represents a condition or situation in which an object exists for a period of time while performing an activity.

UML Notation

```
+-----+
| Read Card |
+-----+
```

Example

- Read Card
- Verify PIN
- Process Transaction

3. Transition

Definition

A **Transition** is the movement of an object **from one state to another** when an event occurs.

UML Notation

State A -----> State B

Example

Read Card -----> Verify PIN

4. Event

Definition

An **Event** is an occurrence that **causes a transition** from one state to another.

Example

- Insert Card
- Enter PIN
- Withdraw Request
- Cancel Transaction

Example:

Waiting ---- Insert Card ----> Read Card

5. Final State

Definition

The **Final State** represents the **termination or completion** of the object's lifecycle.

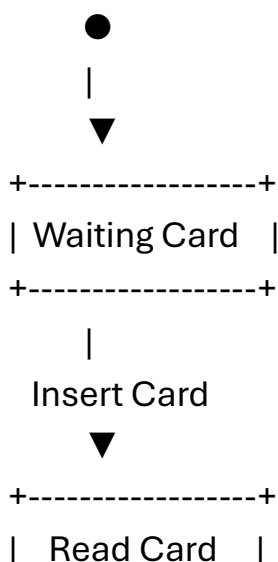
UML Notation

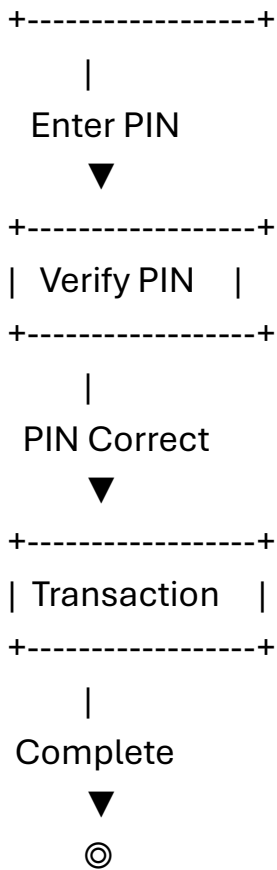
◎

Example

After the ATM ejects the card, the system reaches the **Final State**.

Simple State Diagram Example (ATM)





Advantages of State Diagram

- Shows the dynamic behavior of a system.
- Represents object states clearly.
- Helps understand system workflow.
- Useful for software design and testing.

Conclusion

The main components of a State Diagram are **Initial State, State, Transition, Event, and Final State**. These components help describe how an object changes its state during execution.

Exam Definition (Write First)

A State Diagram is a UML behavioral diagram that shows the different states of an object and the transitions between them during its lifecycle.

Q4 (b) Explain how you model different events in a State Diagram. [5 Marks]

(SPPU BE OOMD – Simple Answer)

Definition

An **Event** is an occurrence that **causes an object to change from one state to another** in a State Diagram. Events trigger **state transitions** and define the dynamic behavior of the system.

In UML, different types of events are used to model different situations.

Types of Events in State Diagram

1. Call Event

Definition

A **Call Event** occurs when an **operation or method is called** on an object.

Example

In an ATM system, when the customer selects **Withdraw Cash**, the operation **withdraw()** is called.

Notation

Idle ----- withdraw() -----> Processing

2. Signal Event

Definition

A **Signal Event** occurs when an object **receives a signal or message** from another object.

Example

A bank server sends an **"Authentication Successful"** signal to the ATM.

Notation

Waiting ----- Signal Received -----> Verified

3. Time Event

Definition

A **Time Event** occurs **after a specified time** or at a particular time.

Example

If the user does not enter the PIN within **30 seconds**, the ATM automatically cancels the transaction.

Notation

Waiting PIN ---- after(30 sec) ----> Cancel Transaction

4. Change Event

Definition

A **Change Event** occurs when a specified **condition becomes true**.

Example

When the **balance becomes less than ₹1000**, the system displays a low balance warning.

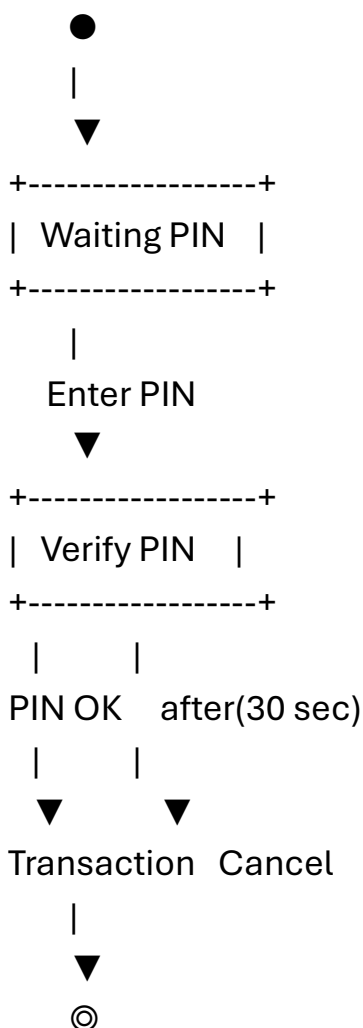
Notation

Account Active ---- [balance < 1000] ----> Low Balance

Summary Table

Event Type	Description	Example
Call Event	Triggered by a method call	withdraw()
Signal Event	Triggered by receiving a signal	Authentication successful
Time Event	Triggered after a time interval	after(30 sec)
Change Event	Triggered when a condition becomes true balance < 1000	

Simple State Diagram Example



Advantages

- Models the dynamic behavior of a system.
 - Clearly shows when state changes occur.
 - Helps in designing event-driven systems.
 - Improves understanding of system workflow.
-

Conclusion

Events are responsible for changing the state of an object in a State Diagram. The main types of events are **Call Event, Signal Event, Time Event, and Change Event**, each used for different situations in object-oriented systems.

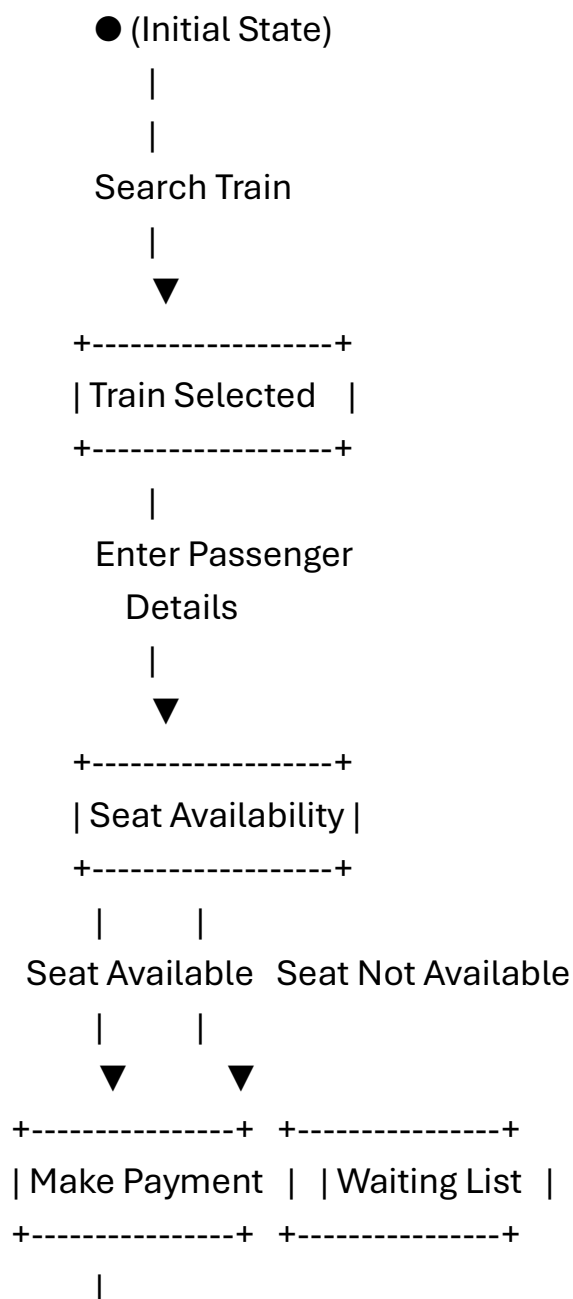
Q4 (c) Draw State Chart Diagram for Railway Reservation System. [5 Marks]

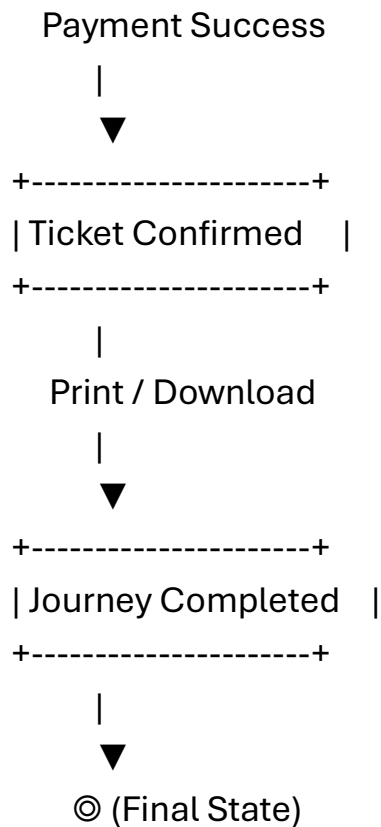
(SPPU BE OOMD – Simple Answer)

Definition

A **State Chart Diagram** represents the different **states** of an object and the **transitions** between those states based on events. It is used to model the dynamic behavior of the Railway Reservation System.

State Chart Diagram





Explanation of States

1. Initial State

- The railway reservation process begins.

2. Search Train

- The user searches for available trains by entering source, destination, and travel date.

3. Train Selected

- The user selects a suitable train from the available list.

4. Enter Passenger Details

- Passenger information such as name, age, and berth preference is entered.

5. Seat Availability

- The system checks whether seats are available.
- **If seats are available:** Proceed to payment.
- **If seats are not available:** The ticket is placed on the waiting list.

6. Make Payment

- The passenger pays the ticket fare using the selected payment method.

7. Ticket Confirmed

- After successful payment, the reservation is confirmed and the ticket is generated.

8. Journey Completed

- The reservation process ends after the journey.

9. Final State

- Indicates the end of the reservation process.

Advantages of State Chart Diagram

- Shows the dynamic behavior of the reservation system.
- Clearly represents state transitions.
- Helps in software design and testing.
- Makes the reservation workflow easy to understand.

Conclusion

The Railway Reservation State Chart Diagram models the complete reservation process from **searching a train** to **ticket confirmation** and **journey completion**, showing all important states and transitions.

Q4 (b) Write Short Notes on: (i) Package (ii) Derived Data (iii) Constraints [5 Marks]

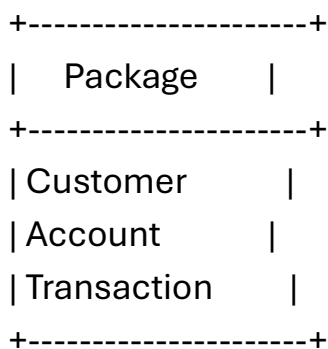
(SPPU BE OOMD – Simple Answer)

i) Package

Definition

A **Package** is a UML element used to **group related classes, interfaces, diagrams, and other model elements** into a single unit. It helps organize large systems into manageable parts.

UML Notation



Advantages

- Organizes related classes.
- Reduces system complexity.
- Improves readability and maintenance.

Example

In a **Library Management System**, classes like **Book**, **Member**, and **Librarian** can be grouped into a package named **Library**.

ii) Derived Data

Definition

Derived Data is data that is **calculated from other attributes** rather than stored directly in the database. In UML, it is represented by placing a **slash (/)** before the attribute name.

UML Notation

Person

dateOfBirth

/age

Example

- **Date of Birth = 15/08/2005**
- **Age = 21 years** (derived from date of birth)

Here, **Age** is derived data.

Advantages

- Avoids storing duplicate data.
- Ensures consistency.
- Reduces storage requirements.

iii) Constraints

Definition

A **Constraint** is a rule or condition that **restricts the values or behavior** of objects in a UML model. Constraints ensure that the system follows specific business rules.

UML Notation

Student

age : int

{age >= 18}

(Constraints are written inside curly braces { }.)

Example

- {balance >= 0}
- {age >= 18}
- {marks <= 100}

Advantages

- Maintains data accuracy.
- Enforces business rules.
- Improves system reliability.

Conclusion

- **Package** groups related UML elements.
- **Derived Data** is calculated from other data and is shown with a / symbol.
- **Constraints** define rules or conditions and are written inside { }.

Q4 (c) Define Event and explain different types of Events. [5 Marks]

(SPPU BE OOMD – Simple Answer)

Definition of Event

An **Event** is an occurrence or action that **causes a change in the state of an object**. When an event occurs, the object may move from one state to another in a **State Diagram**.

Example: In an ATM system, **Insert Card** is an event that changes the ATM from the **Idle** state to the **Read Card** state.

Types of Events

1. Call Event

Definition

A **Call Event** occurs when an operation or method is called on an object.

Example

- withdraw()
- deposit()

In an ATM, calling **withdraw()** starts the withdrawal process.

2. Signal Event

Definition

A **Signal Event** occurs when one object sends a signal or message to another object.

Example

A bank server sends a **Transaction Successful** message to the ATM.

3. Time Event

Definition

A **Time Event** occurs after a specified time or at a particular time.

Example

If no operation is performed for **30 seconds**, the ATM automatically logs out the user.

4. Change Event

Definition

A **Change Event** occurs when the value of a condition or attribute changes.

Example

When the **account balance becomes zero**, the account status changes.

Summary Table

Type of Event Description		Example
Call Event	Triggered by calling a method	withdraw()
Signal Event	Triggered by receiving a signal/message	Transaction successful
Time Event	Triggered after a specific time	Session timeout after 30 sec
Change Event	Triggered when a condition changes	Balance becomes zero

Advantages of Events

- Control state transitions.
- Represent system behavior dynamically.
- Help model real-world processes.
- Improve understanding of system workflow.

Q. What do you mean by Association Ends? Explain N-ary Association in short. [5 Marks]

(SPPU BE OOMD – Simple Answer)

i) Association Ends

Definition

Association Ends are the endpoints of an association relationship that connect two classes in a UML class diagram. They define the **role, multiplicity, and navigability** of each class involved in the association.

Each end of an association specifies how one class is related to another class.

Components of Association End

1. Role Name

- Defines the role played by a class in an association.
- Helps identify how one class interacts with another.

Example:

Customer ----- Account
owner account

2. Multiplicity

- Specifies the number of objects that can participate in an association.

Common multiplicities:

Multiplicity Meaning

1	Exactly one
0..1	Zero or one
*	Many
1..*	One or more

Example:

Customer 1 ----- * Account

One customer can have many accounts.

3. Navigability

- Shows the direction in which one class can access another class.
- Represented using an arrow.

Example:

Customer -----> Account

Customer can access Account details.

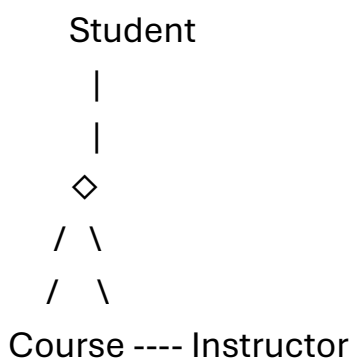
ii) N-ary Association

Definition

An **N-ary Association** is an association that involves **three or more classes simultaneously**. It represents a relationship where a single association cannot be represented using only two classes.

In UML, an N-ary association is represented by a **diamond symbol connected to all participating classes**.

UML Notation



Example

University Course Registration System

Classes:

- Student
- Course

- Instructor

Relationship:

- A student registers for a course taught by an instructor.

Here, all three classes participate in one association.

Advantages of N-ary Association

- Represents complex relationships.
- Avoids incorrect modeling of multiple two-way associations.
- Provides a clear view of relationships among multiple classes.

Difference Between Binary and N-ary Association

Binary Association	N-ary Association
Involves two classes	Involves three or more classes
Simple relationship	Complex relationship
Example: Student–Course	Example: Student–Course–Instructor

Conclusion

Association Ends define the roles and properties of classes involved in an association, while an **N-ary Association** represents a relationship among three or more classes in UML.

Q. What is State? Explain with Example, Transitions and Conditions. [5 Marks]

(SPPU BE OOMD – Simple Answer)

Definition of State

A **State** represents a condition or situation in which an object exists during its lifetime.

It describes the **behavior and status of an object at a particular point of time**.

An object changes from one state to another when an **event occurs** and the specified **condition is satisfied**.

UML Notation of State

```
+-----+
| State |
+-----+
```

Example of State

ATM System

An ATM object can have different states:

- Idle

- Card Inserted
- PIN Verification
- Transaction Processing
- Cash Dispensed

Example:

Idle → Card Inserted → Verify PIN → Transaction

Each stage represents a different state of the ATM object.

1) Transition

Definition

A **Transition** is the movement of an object from one state to another state due to the occurrence of an event.

Notation

State A -----> State B

Event

Example:

Idle -----Insert Card-----> Card Inserted

Here, **Insert Card** is the event that causes the transition from **Idle** state to **Card Inserted** state.

2) Condition (Guard Condition)

Definition

A **Condition** is a Boolean expression that must be true for a transition to occur. It is written inside square brackets [] in UML.

Notation

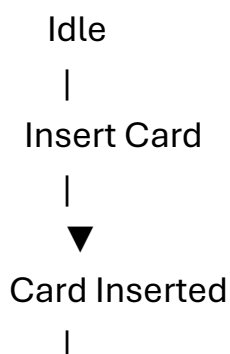
State A ---- Event [Condition] ----> State B

Example:

Verify PIN ---- Enter PIN [PIN Correct] ----> Transaction

- If PIN is correct → Transaction state starts.
- If PIN is incorrect → User is asked to enter PIN again.

Complete Example (ATM State Transition)



Enter PIN [Correct]



Transaction



Withdraw Cash



Cash Dispensed

Difference Between Transition and Condition

Transition

Shows movement between states

Triggered by an event

Example: Insert Card

Condition

Shows a rule for movement

Must be satisfied before transition

Example: PIN is correct

Conclusion

A **State** represents the current condition of an object. A **Transition** defines the movement between states, while a **Condition** controls whether the transition can occur.

Q3 (a) Construct State Transition Diagram for Computerization of College Library. [6 Marks]

(SPPU BE OOMD – Simple Answer)

Definition

A **State Transition Diagram** represents the different states of an object and the transitions between states caused by events. It describes the dynamic behavior of a system.

State Transition Diagram for College Library System

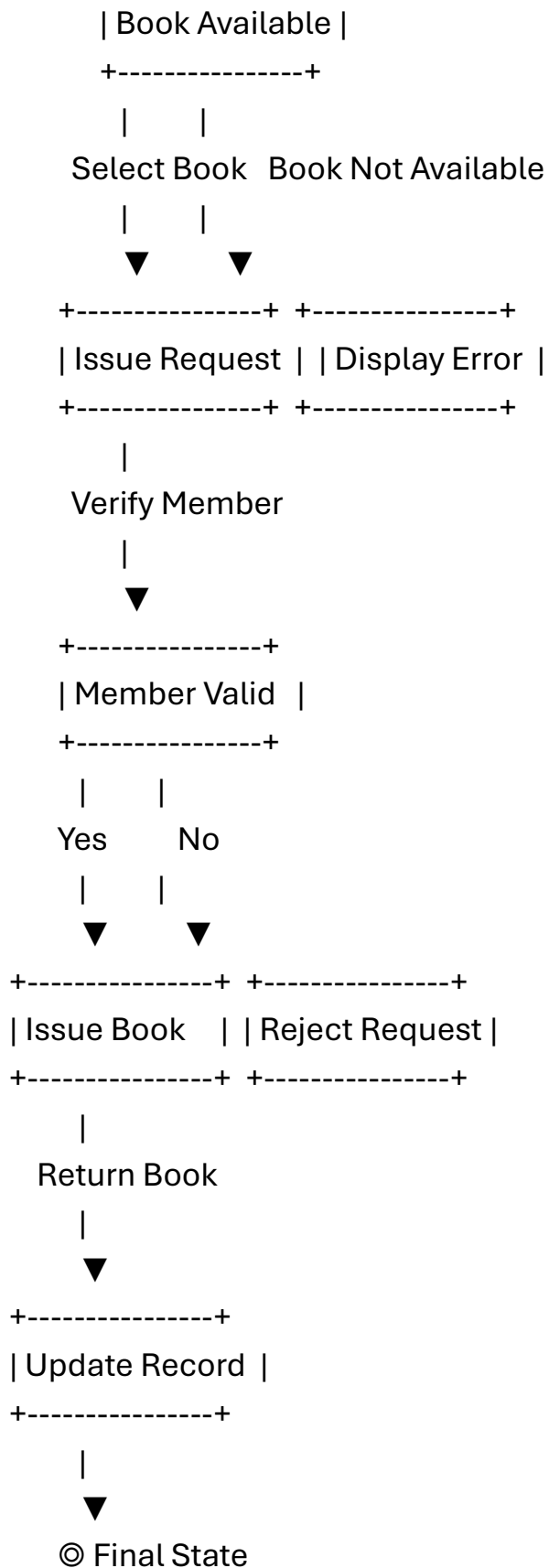
● Initial State



Search Book



+-----+



Explanation of States

1. Initial State

- Library system starts from this state.

2. Search Book

- Student searches for required books.

3. Book Available

- System checks availability of the selected book.

4. Issue Request

- Student requests to issue the book.

5. Verify Member

- System checks whether the student is a valid library member.

6. Issue Book

- Book is issued and database records are updated.

7. Return Book

- Student returns the issued book.

8. Update Record

- Library system updates book availability.
-

Q3 (b) Explain Different Pseudo States in State Chart Diagram. [6 Marks]

Definition

Pseudo States are special states in a UML State Chart Diagram that control the flow of transitions but do not represent actual states of an object.

Types of Pseudo States

1. Initial State

Definition

The initial state indicates the starting point of a state machine.

Notation



Example

Library system starts from "Search Book".

2. Final State

Definition

The final state represents the completion or termination of a state machine.

Notation



Example

Transaction completed.

3. Choice State

Definition

Choice state is used to represent conditional branching where one of multiple paths is selected based on conditions.

Notation



Example

Check Availability



Available Not Available

4. Junction State

Definition

A junction state is used to merge or split multiple transitions into a single path.

Notation

● (small filled circle)

Example

Combining different transaction paths.

5. Fork State

Definition

A fork state divides one transition into multiple parallel transitions.

Example

ATM may perform:

- Update account
- Print receipt

simultaneously.

6. Join State

Definition

A join state combines multiple parallel transitions into one transition.

Example

After updating account and printing receipt, the process continues.

Q3 (c) Differentiate between Aggregation and Composition. [6 Marks]

Definition

Aggregation

Aggregation is a **weak "Has-A" relationship** between classes where the child object can exist independently of the parent object.

Composition

Composition is a **strong "Has-A" relationship** where the child object cannot exist without the parent object.

Difference Between Aggregation and Composition

Aggregation

It is a weak relationship.

Child object can exist independently.

Parent and child have separate lifecycles.

Deleting parent does not delete child.

Represented using a **hollow diamond (◇)**.

Example: Department and Professor.

Composition

It is a strong relationship.

Child object cannot exist independently.

Parent and child have the same lifecycle.

Deleting parent also deletes child.

Represented using a **filled diamond (◆)**.

Example: House and Room.

UML Notation

Aggregation

Department ◇----- Professor

Composition

House ◆----- Room

Examples Explanation

Aggregation Example:

Department – Professor

- A department has professors.
- If a department is removed, professors can still exist.

Composition Example:

House – Room

- A house contains rooms.
 - If the house is destroyed, rooms also cease to exist.
-

Conclusion

Aggregation represents **independent ownership**, while Composition represents **strong ownership and dependency** between objects.

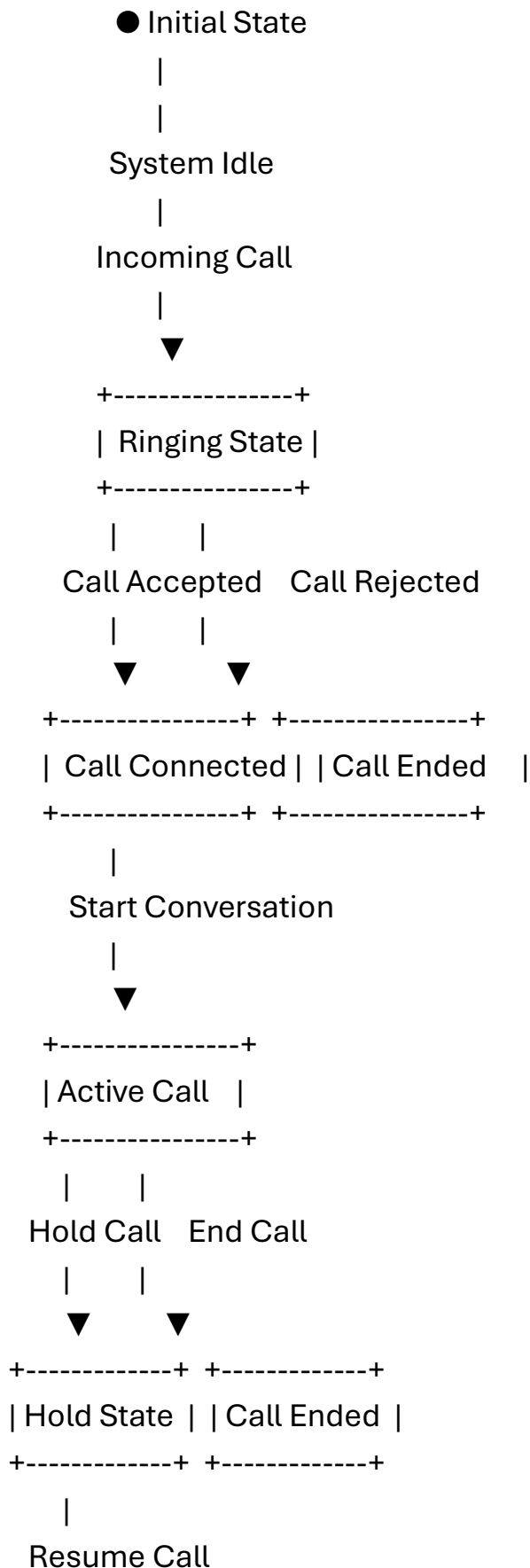
Q. Draw State Chart Diagram for Handling Telecommunication System. [5 Marks]

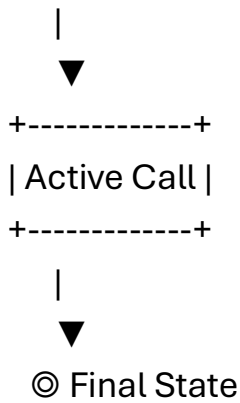
(SPPU BE OOMD – Simple Answer)

Definition

A **State Chart Diagram** represents the different states of an object and the transitions between these states based on events. It is used to model the dynamic behavior of a telecommunication system.

State Chart Diagram for Telecommunication System





Explanation of States

1. Initial State

- Represents the starting point of the telecommunication system.

2. Idle State

- The phone is ready to receive or make calls.

3. Ringing State

- System receives an incoming call and alerts the user.

4. Call Connected State

- The call is accepted and communication starts.

5. Active Call State

- Users are communicating with each other.

6. Hold State

- The ongoing call is temporarily paused.

7. Call Ended State

- The call is terminated after the user disconnects.

8. Final State

- Represents the completion of the call process.
-

Events/Transitions

Event	Transition
Incoming call	Idle → Ringing
Accept call	Ringing → Call Connected
Reject call	Ringing → Call Ended
Hold	Active Call → Hold
Resume	Hold → Active Call
Disconnect	Active Call → Call Ended

Conclusion

The telecommunication system state chart diagram shows the complete call handling process from **idle state** → **incoming call** → **connection** → **conversation** → **termination**. It helps in understanding the dynamic behavior of the system.

Q (b) What is State? Explain with Example, Transition and Condition. [5 Marks]

(SPPU BE OOMD – Simple Answer)

Definition of State

A **State** represents the condition or situation of an object at a particular point of time during its life cycle. It describes the behavior of an object when it responds to events. A state remains active until an event causes a **transition** to another state.

UML Notation of State

```
+-----+
| State |
+-----+
```

Example: ATM System

An ATM object can have different states:

Idle → Card Inserted → PIN Verification → Transaction → Cash Dispensed

Explanation:

- **Idle:** ATM is waiting for a customer.
 - **Card Inserted:** Customer inserts ATM card.
 - **PIN Verification:** System checks PIN.
 - **Transaction:** User performs withdrawal/deposit.
 - **Cash Dispensed:** ATM provides cash.
-

Transition

Definition

A **Transition** is the movement of an object from one state to another state due to an event.

Notation:

State A ----- Event -----> State B

Example:

Idle ---- Insert Card ----> Card Inserted

Here, **Insert Card** is the event causing the transition.

Condition (Guard Condition)

Definition

A **Condition** is a Boolean expression that must be true for a transition to occur. It is written inside square brackets [].

Notation:

State A ---- Event [Condition] ----> State B

Example:

Verify PIN ---- Enter PIN [PIN Correct] ----> Transaction

- If PIN is correct → Transaction starts.
- If PIN is incorrect → User is asked to enter PIN again.

Conclusion

A **State** defines the current condition of an object, **Transition** represents movement between states, and **Condition** controls whether the transition can occur.

Q (c) Differentiate between Procedural Language and Object-Oriented Language. [6 Marks]**Definition****Procedural Language**

A **procedural language** is a programming approach in which a program is divided into **functions or procedures** that operate on data.

Object-Oriented Language

An **object-oriented language** is a programming approach in which a program is organized around **objects and classes** that contain data and methods.

Difference Between Procedural and Object-Oriented Language

Procedural Language	Object-Oriented Language
Program is divided into functions/procedures.	Program is divided into objects and classes.
Focus is on functions and procedures .	Focus is on data and objects .
Data is usually separate from functions.	Data and methods are combined using encapsulation.
Follows a top-down approach .	Follows a bottom-up approach .
Provides less data security.	Provides better data security using encapsulation.
Code reusability is limited.	Supports code reusability through inheritance.
Difficult to maintain large programs.	Easier to maintain and modify large programs.
Examples: C, Pascal, FORTRAN	Examples: Java, C++, Python, C#

Example

Procedural Approach (C)

```
main()
{
    input();
    calculate();
    display();
}
```

Object-Oriented Approach

Student

data

methods

The object **Student** contains both data and functions.

Conclusion

Procedural languages focus on **functions and procedures**, whereas object-oriented languages focus on **objects and classes**. OOP provides better **security, reusability, and maintainability** for large software systems.